

Universidad Autónoma de Aguascalientes



Proyecto segundo parcial

Visión robótica

Nombre del Docente:

Carlos Alberto López Hernández

Nombres del Equipo:

Leonardo Miguel Sánchez Lupercio

ID: 264993

Alan Dai Vázquez Martínez

ID: 349659

Ruth Anahi Diaz Valderrama

ID: 213580

Nancy Carolina García Delgado

ID: 334317

Objetivo

Crear una librería para Python la cual nos permita identificar líneas, esquinas e intersecciones en planos arquitectónicos.

Introducción

El procesamiento digital de imágenes constituye una de las áreas más importantes dentro de la visión por computadora, ya que permite analizar y extraer información relevante a partir de imágenes digitales. En el ámbito de la arquitectura e ingeniería, los planos arquitectónicos representan una fuente fundamental de información gráfica, debido a que contienen elementos geométricos como líneas, esquinas e intersecciones que describen estructuras y distribuciones espaciales. Sin embargo, cuando estos documentos son digitalizados mediante escaneo o fotografía, suelen presentar problemas como ruido, inclinación, variaciones de iluminación y bajo contraste, lo que dificulta su análisis automático.

Mediante la creación de una librería podemos integrar diferentes funciones las cuales nos permitirán procesar las imágenes con el fin de cumplir con los objetivos del proyecto, permitiéndonos mostrar de manera simple el resultado del procesamiento.

Marco Teórico

Las librerías Numpy y CV2 es esencial para el desarrollo del proyecto, debido a que son de las herramientas más simples para el procesamiento de imágenes.

- Transformación de color de la imagen: cambiando los canales de la imagen podemos procesarla de diferente manera, siendo una de las más útiles el cambiar a escala de grises la imagen.
- Mascara gaussiana: el desenfoque gaussiano permite eliminar el ruido de la imagen entregándonos una imagen suavizada.
- Adaptive threshold: Modifica la brecha entre los distintos valores de intensidad de la imagen, permitiéndonos aumentarlo o disminuirlo para el mejor procesamiento de la imagen.
- Copias: nos permite crear una copia de la imagen original con el fin de no alterar esta al momento de procesarlo
- Transformada de Hough: es una técnica de identificación de patrones lineales en una imagen binaria.
- CornerHarris: Detecta esquinas e intersecciones.

Desarrollo

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def preparar_imagen(ruta_imagen):
```

```
    """Carga la imagen y devuelve la versión RGB y Gris."""
```

```
    img = cv2.imread(ruta_imagen)
```

```
    if img is None:
```

```
        return None, None, None
```

```
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
    gris = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
```

```
    return img, img_rgb, gris
```

```
def obtener_mascara_muros(gris, k_size=6, b_size=5):
```

```
    """Procesa la imagen gris para obtener la máscara de muros."""
```

```
    desenfocado = cv2.GaussianBlur(gris, (b_size, b_size), 0)
```

```
    binaria = cv2.adaptiveThreshold(
```

```
        desenfocado, 255,
```

```
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
```

```
        cv2.THRESH_BINARY_INV, 11, 2
```

```
    )
```

```
    kernel = np.ones((k_size, k_size), np.uint8)
```

```
    muros_limpios = cv2.morphologyEx(binaria, cv2.MORPH_OPEN, kernel)
```

```
    return muros_limpios
```

```
def detectar_detalles(img_rgb, muros_limpios, l_thresh=40):
```

```
    """Dibuja líneas y esquinas sobre la imagen y la devuelve."""
```

```
    resultado = img_rgb.copy()
```

```

# Líneas

lineas = cv2.HoughLinesP(muros_limpios, 1, np.pi/180, L_thresh, minLineLength=10,
maxLineGap=10)

if lineas is not None:

    for l in lineas:

        x1, y1, x2, y2 = l[0]

        cv2.line(resultado, (x1, y1), (x2, y2), (0, 255, 0), 2)


# Esquinas

muros_float = np.float32(muros_limpios)

esquinas = cv2.cornerHarris(muros_float, 2, 3, 0.04)

esquinas = cv2.dilate(esquinas, None)

resultado[esquinas > 0.01 * esquinas.max()] = [255, 0, 0]


return resultado


def mostrar(gris, mascara, final):

    """Muestra la comparativa de imágenes."""

    plt.figure(figsize=(15, 8))

    imagenes = [gris, mascara, final]

    titulos = ["Gris", "Máscara", "Resultado"]

    for i in range(3):

        plt.subplot(1, 3, i+1)

        plt.imshow(imagenes[i], cmap='gray' if i < 2 else None)

        plt.title(titulos[i])

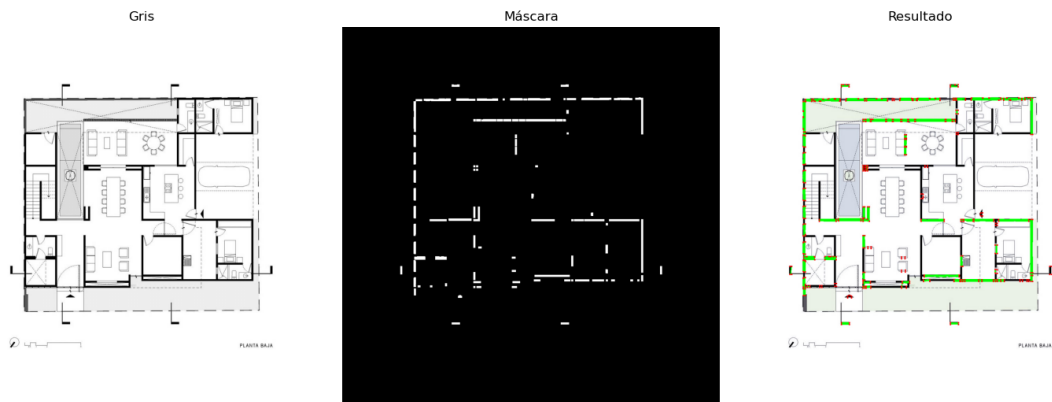
        plt.axis('off')

```

```
plt.tight_layout()
```

```
plt.show()
```

Resultados



Conclusiones

El código implementa un flujo completo de procesamiento digital de imágenes que combina técnicas como filtrado espacial, umbralización, operaciones morfológicas, detección de

líneas y detección de esquinas, con el objetivo de extraer información estructural relevante a partir de planos arquitectónicos digitalizados. Mediante estas técnicas es posible identificar elementos geométricos importantes, como muros, vértices e intersecciones, facilitando el análisis automático de los planos.

La integración de estos métodos permite desarrollar aplicaciones con utilidad práctica en áreas de visión por computadora y análisis arquitectónico, entre las que destacan:

- Automatización en la interpretación de planos arquitectónicos.
- Identificación de muros y puntos críticos dentro del diseño estructural.
- Extracción de características geométricas para sistemas CAD y simulaciones.
- Generación de información visual útil para procesos de análisis y digitalización.

Referencias

Zyprian, F., & Zyprian, F. (2025, 17 diciembre). *OpenCV – cv2 Guide für Python Entwickler*. Konfuzio. <https://konfuzio.com/es/cv2/>

<https://aprendeconalf.es/docencia/python/manual/numpy/>